SDK Core Concepts

This guide explains the fundamental concepts for working with the Graph OLAP SDK - the **sole user interface** for the Graph OLAP Platform. All platform operations are performed through this SDK in Jupyter notebooks. This document covers platform architecture, resource hierarchy, connection lifecycle, and query execution patterns.

---

1. Platform Architecture Overview

The Graph OLAP Platform follows a **control plane + data plane** architecture where the Control Plane manages resource lifecycle and the Data Plane (Wrapper Pods) handles graph operations.

Architecture Diagram

```
                                  +------------------+
                                  |   Jupyter/SDK    |
                                  |    (Client)      |
                                  +--------+---------+
                                           |
                                           | HTTPS
                                           v
+-------------------------------------------------------------------------+
|                           Ingress Controller                             |
|   /api/* -> Control Plane  |  /{instance-id}/* -> Wrapper Pod           |
+-------------------------------------------------------------------------+
              |                                          |
              v                                          v
+------------------------+                   +------------------------+
|    Control Plane       |                   |    Wrapper Pod (N)     |
|    (FastAPI)           |                   |    (FastAPI + Graph)   |
|                        |                   |                        |
|  - REST API            |                   |  - Cypher Queries      |
|  - Resource Lifecycle  |                   |  - Graph Algorithms    |
|  - Background Jobs     |                   |  - NetworkX Support    |
+----------+-------------+                   +------------------------+
           |                                            |
           v                                            v
+------------------------+                   +------------------------+
|    Cloud SQL           |                   |    Google Cloud        |
```

```
|    (PostgreSQL)        |                    |    Storage (Parquet)    |
+------------------------+                    +-------------------------+
```

Component Responsibilities

Component	Role	SDK Interaction
Control Plane	Manages mappings, snapshots, and instances	SDK calls `/api/*` endpoints
Wrapper Pods	Runs graph database with data	SDK calls `/{id}/*` for queries
Export Workers	Exports SQL results to Parquet	Background (no SDK interaction)
Cloud SQL	Stores platform metadata	Internal (no SDK interaction)
GCS	Stores exported Parquet files	Internal (no SDK interaction)

Data Flow:

1. User creates a **Mapping** defining SQL-to-graph schema via Control Plane
2. User creates an **Instance** from a Mapping; system automatically creates a snapshot internally
3. Export Workers run UNLOAD queries to GCS (automatic)
4. Wrapper Pod loads Parquet data when snapshot is ready (automatic)
5. User connects to the Instance and executes Cypher queries/algorithms

SDK as Client Interface

The SDK (`graph-olap` package) provides a unified client interface that abstracts API complexity, manages authentication, provides type safety with Pydantic models, and enables rich Jupyter display.

```python
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType
```

```python
client = GraphOLAPClient.from_env()

# Control Plane operations
mapping = client.mappings.get(1)

# Create instance directly from mapping (snapshot managed internally)
instance = client.instances.create_from_mapping_and_wait(
    mapping_id=1,
    name="Analysis",
    wrapper_type=WrapperType.RYUGRAPH,
)

# Data Plane operations (via connection)
conn = client.instances.connect(instance.id)
result = conn.query("MATCH (n) RETURN count(n)")
```

---

2. Resource Hierarchy

The platform has three core resources: **Mapping**, **Snapshot** (internal), and **Instance**.

> **Note:** Snapshots are now managed internally. Users create instances directly from mappings.

```
Mapping (Schema Definition)
    +-- MappingVersion v1, v2, ...
    |       +-- NodeDefinition: Customer, Product
    |       +-- EdgeDefinition: PURCHASED
    +-- Instance (Running Graph Database)
            +-- Snapshot (managed internally)
                    +-- Parquet files in GCS
```

Mappings

A **Mapping** defines how Starburst SQL queries map to graph nodes and edges.

Key Characteristics:

- Owned by a user (`owner_username`)
- Contains multiple immutable **versions**
- Supports lifecycle settings (TTL, inactivity timeout)

Versioning: Each update creates a new **MappingVersion**. Versions are immutable, ensuring snapshots reference specific versions and changes are tracked.

```
# Get a mapping and its versions
mapping = client.mappings.get(mapping_id=1)
versions = client.mappings.list_versions(mapping_id=1)

# Compare versions
diff = client.mappings.diff(mapping_id=1, from_version=1, to_version=2)
print(f"Nodes added: {diff.summary['nodes_added']}")
```

NodeDefinition

Specifies how to create graph nodes from SQL:

Field	Description	Example
`label`	Node type label	`"Customer"`
`sql`	SQL query to extract data	`"SELECT id, name FROM customers"`
`primary_key`	Unique identifier	`{"name": "id", "type": "STRING"}`
`properties`	Additional properties	`[{"name": "name", "type": "STRING"}]`

```
from graph_olap.models import NodeDefinition, PropertyDefinition

customer_node = NodeDefinition(
    label="Customer",
    sql="SELECT customer_id, name, email FROM analytics.customers",
    primary_key={"name": "customer_id", "type": "STRING"},
    properties=[
        PropertyDefinition(name="name", type="STRING"),
        PropertyDefinition(name="email", type="STRING"),
    ],
)
```

EdgeDefinition

Specifies how to create relationships between nodes:

Field	Description	Example
`type`	Relationship type	`"PURCHASED"`
`from_node` / `to_node`	Source/target node labels	`"Customer"`, `"Product"`
`sql`	SQL query for relationship data	`"SELECT customer_id, product_id FROM orders"`
`from_key` / `to_key`	Foreign keys	`"customer_id"`, `"product_id"`
`properties`	Edge properties	`[{"name": "quantity", "type": "INT64"}]`

```python
from graph_olap.models import EdgeDefinition

purchased_edge = EdgeDefinition(
    type="PURCHASED",
    from_node="Customer",
    to_node="Product",
    sql="SELECT customer_id, product_id, quantity FROM orders",
    from_key="customer_id",
    to_key="product_id",
    properties=[PropertyDefinition(name="quantity", type="INT64")],
)
```

Supported Property Types

Type	Python Equivalent
`STRING`	`str`
`INT64`	`int`
`DOUBLE`	`float`
`BOOL`	`bool`

Type	Python Equivalent
`DATE`	`datetime.date`
`TIMESTAMP`	`datetime.datetime`

Snapshots

> **SNAPSHOT FUNCTIONALITY DISABLED**
>
> Explicit snapshot APIs have been disabled. Instances are now created directly from mappings without requiring explicit snapshot creation. The snapshot layer operates implicitly when instances are created.
>
> Use `client.instances.create_from_mapping()` or `client.instances.create_from_mapping_and_wait()` instead of the snapshot methods described below.

A **Snapshot** is a point-in-time export of data from Starburst based on a specific mapping version. Snapshots are now managed internally when creating instances from mappings.

Status Lifecycle (Internal)

```
pending --> creating --> ready
                    \--> failed --> creating (retry)
```

Status	Description	Can Create Instance?
`pending`	Waiting for export workers	No
`creating`	UNLOAD queries executing	No
`ready`	All Parquet files written	Yes
`failed`	Export encountered errors	No

Recommended: Create Instance Directly from Mapping

```python
from graph_olap_schemas import WrapperType

# Recommended approach - creates snapshot automatically
instance = client.instances.create_from_mapping_and_wait(
    mapping_id=1,
    name="My Analysis",
    wrapper_type=WrapperType.RYUGRAPH,
    ttl=24,  # hours
)

# Instance is ready - snapshot was managed internally
conn = client.instances.connect(instance.id)
```

Instances

An **Instance** is a running graph database loaded from a snapshot.

Status Lifecycle

```
starting --> running --> stopping --> [deleted]
        \--> failed
```

Status	Description	Can Query?
`starting`	Pod initializing, loading data	No
`running`	Ready for queries and algorithms	Yes
`stopping`	Graceful shutdown in progress	No
`failed`	Error during startup	No

Creating Instances

```python
from graph_olap_schemas import WrapperType

# Create directly from mapping (recommended) - snapshot managed internally
instance = client.instances.create_from_mapping_and_wait(
    mapping_id=mapping.id,
    name="Analysis Instance",
```

```
    wrapper_type=WrapperType.RYUGRAPH,  # or WrapperType.FALKORDB
    ttl=24,  # 24-hour time-to-live
    inactivity_timeout=2,  # 2-hour idle timeout
)
```

Locking Mechanism

Instances use an **exclusive lock** for algorithm execution:

- **Queries**: Always allowed concurrently (read-only)
- **Algorithms**: Require exclusive lock (one at a time)

```
lock = conn.get_lock()
if lock.locked:
    print(f"Locked by: {lock.holder_name}, running: {lock.algorithm}")
```

TTL and Lifecycle

Setting	Description	Format
`ttl`	Time-to-live from creation	ISO 8601 (e.g., `"PT24H"`)
`inactivity_timeout`	Idle time before cleanup	ISO 8601 (e.g., `"PT2H"`)

```
# Update lifecycle
instance = client.instances.set_lifecycle(instance_id, ttl="PT12H")

# Extend TTL from current expiry
instance = client.instances.extend_ttl(instance_id, hours=24)
```

---

3. Connection Lifecycle

The SDK uses a two-phase connection model: **client initialization** for Control Plane operations and **instance connection** for Data Plane operations.

Client Initialization

```python
from graph_olap import GraphOLAPClient

# From environment variables (recommended)
client = GraphOLAPClient.from_env()

# Or explicit configuration
client = GraphOLAPClient(
    api_url="https://graph-olap.example.com",
    api_key="sk-xxx",
    timeout=30.0,
    max_retries=3,
)
```

Context Manager Pattern (Recommended)

```python
with GraphOLAPClient.from_env() as client:
    # Create instance directly from mapping (snapshot managed internally)
    instance = client.instances.create_from_mapping_and_wait(
        mapping_id=1,
        name="Analysis",
        wrapper_type=WrapperType.RYUGRAPH,
    )
    conn = client.instances.connect(instance.id)
    result = conn.query("MATCH (n) RETURN count(n)")
    client.instances.terminate(instance.id)
# Client closed automatically
```

Instance Connection

Once an instance is running, connect to it for queries and algorithms:

```python
# Get connection to running instance
conn = client.instances.connect(instance_id)

# Connection properties
print(f"Instance ID: {conn.id}")
print(f"Instance Name: {conn.name}")
print(f"Snapshot ID: {conn.snapshot_id}")
print(f"Status: {conn.current_status}")

# Execute queries
result = conn.query("MATCH (n:Customer) RETURN n LIMIT 10")
```

```
# Close when done
conn.close()
```

Connection Context Manager (recommended):

```
with client.instances.connect(instance_id) as conn:
    df = conn.query_df("MATCH (n:Customer) RETURN n.name, n.email")
    # Connection automatically closed after block
```

Quick Start Pattern

For rapid prototyping:

```
# Create snapshot, instance, and connect in one call
conn = client.quick_start(
    mapping_id=1,
    wrapper_type=WrapperType.RYUGRAPH,
)
result = conn.query("MATCH (n) RETURN count(n)")
# Remember to terminate the instance when done!
```

---

4. Query Execution

The SDK provides multiple methods for executing Cypher queries.

Query Methods Overview

Method	Returns	Best For
`query()`	`QueryResult`	Flexible access to results
`query_df()`	DataFrame	Data analysis workflows
`query_scalar()`	Single value	Counts and aggregations
`query_one()`	Dict or None	Single record lookup

query() - Structured Results

```
result = conn.query("""
    MATCH (c:Customer)-[:PURCHASED]->(p:Product)
    RETURN c.name AS customer, p.name AS product
    LIMIT 100
""")

# Access metadata
print(f"Columns: {result.columns}, Rows: {result.row_count}")

# Iterate over rows
for row in result:
    print(f"{row['customer']} bought {row['product']}")

# Convert to DataFrame
df = result.to_polars()  # or result.to_pandas()
```

query_df() - DataFrame Results

```
# Polars DataFrame (default)
df = conn.query_df("MATCH (c:Customer) RETURN c.id, c.name")

# Pandas DataFrame
df = conn.query_df("MATCH (c:Customer) RETURN c.*", backend="pandas")
```

query_scalar() - Single Value

```
count = conn.query_scalar("MATCH (n:Customer) RETURN count(n)")
avg = conn.query_scalar("MATCH (c:Customer) RETURN avg(c.total_purchases)")
```

query_one() - Single Record

```
customer = conn.query_one(
    "MATCH (c:Customer {id: $id}) RETURN c.*",
    {"id": "C001"}
)
if customer:
    print(f"Name: {customer['name']}")
```

Parameter Substitution

Use parameters to safely inject values:

```
result = conn.query(
    """
    MATCH (c:Customer)
    WHERE c.total_purchases > $min_purchases
    RETURN c.name, c.total_purchases
    LIMIT $limit
    """,
    parameters={"min_purchases": 1000, "limit": 50}
)
```

Parameter Types:

Python Type	Cypher Type
`str`	String
`int`	Integer
`float`	Float
`bool`	Boolean
`list`	List
`dict`	Map
`None`	Null

Timeout Configuration

```
# Per-query timeout
result = conn.query(
    "MATCH (n)-[*1..5]-(m) RETURN count(*)",
    timeout=120.0,  # 2 minutes
)
```

Error Handling

```python
from graph_olap.exceptions import QueryError, QueryTimeoutError,
ValidationError

try:
    result = conn.query("MATCH (n) RETURN n.invalid")
except QueryTimeoutError:
    print("Query timed out")
except QueryError as e:
    print(f"Query failed: {e}")
```

Schema Inspection

```python
schema = conn.get_schema()

for label, props in schema.node_labels.items():
    print(f":{label} - {[p['name'] for p in props]}")

for rel_type, info in schema.relationship_types.items():
    print(f"[:{rel_type}]")
```

Summary

Concept	Description
Architecture	Control Plane manages resources; Data Plane executes queries
Mapping	Schema definition with SQL-to-graph mappings and versioning
Snapshot	Point-in-time data export (pending -> creating -> ready)
Instance	Running graph database with locking for algorithms
Client	`GraphOLAPClient` for Control Plane operations
Connection	`InstanceConnection` for Data Plane queries

Next Steps:

- 03-algorithms.manual.md - Running graph algorithms

- 04-advanced-patterns.manual.md - Complex workflows
- 05-troubleshooting.manual.md - Debugging tips